

スキルシート

- このスキルシートのリンク - <https://github.com/sugurutakahashi-1234/skills-sheet>

基本情報

- 所在: 東京
- 現在のポジション: AI コンサルタント / FDE / CTO
 - AI コンサルタント: 企業の AI 活用の相談・助言と、案件獲得の営業
 - FDE: 顧客の業務に入り込み、AI ツールを自ら実装して導入まで担当
 - CTO: 自社プロダクト開発・社内 IT 業務全般
- エンジニア歴: 10年目（2017年～）
- 外部リンク:
 - GitHub - <https://github.com/sugurutakahashi-1234>
 - X - https://x.com/suguru_takaha4
 - Qiita - <https://qiita.com/sugurutakahashi12345>
 - Zenn - <https://zenn.dev/ikuraikura>

サマリー

- エンジニア歴 10年目。Web・モバイルのフロントエンドからサーバーサイド、マルチクラウド・IaC までの実装経験あり
- 現在は創業フェーズの [株式会社ZENSHIN](#) の CTO
- AI コンサルタント / FDE として、案件獲得の営業・商談から AI 活用の提案、顧客向け AI ツールの実装・導入までを一気通貫で担当
- 自社では AI マッチングシステム（RAG・MCP・AI エージェント）を toC・toB・社内向けのマルチプロダクト構成で単独開発・運用

強み

- AI コンサルティング～FDE**
 - 経営層から実装担当者まで、相手に合わせた AI 活用の相談・提案を担当
 - 案件獲得の営業・商談から、顧客業務に入り込んだ AI ツールの実装・導入までを一気通貫で担当
- AI システム開発（RAG / MCP / AI エージェント）**
 - 人材マッチングや開発案件の自動評価のシステムを、設計から実運用まで単独でフルスタック開発
 - 非エンジニアの営業が Claude Code / Codex で実業務を回せる状態まで導入
 - AI 判定と人間判断の乖離を本番データで分析し、プロンプトを継続改善する運用ループを確立
- 0 → 1 リードエンジニア経験（5件）**
 - アーキテクチャ選定・CI/CD 構築・チーム運用の設計など、プロジェクト立ち上げの技術リードを担当（現職 CTO でのプロダクト立ち上げを含む）
- KPI 起点のグロース施策**
 - ビジネス KPI（ARPU・LTV・CTR）を基点に、施策の企画・A/B テスト検証・実装までを一気通貫で推進
- フルスタック実装力**
 - iOS・Android・Web のフロントエンド（Swift / React Native / Flutter / Next.js / Astro）からサーバーサイドまで単独で実装可能
 - マルチクラウドの IaC（Terraform × GCP / Cloudflare / AWS）まで一人で構築・運用
- OSS 開発・技術発信**
 - 自作 OSS（TypeScript 製 CLI ツール）を npm / Homebrew / GitHub Actions Marketplace で配布

- 独自 iOS アーキテクチャ「FIA」の考案・公開や、Zenn での技術記事発信を継続

技術スタック

- **AI システム開発**
 - RAG / Embedding / ベクトル検索 (Cloudflare Vectorize, Workers AI)
 - MCP サーバー開発 (@modelcontextprotocol/sdk v2, Better Auth OAuth), WebMCP
 - Vision LLM, Cloudflare Workflows / Queues
 - ユースケース別 LLM モデル選定 (Kimi K2 / gpt-oss / GLM / Llama, 自作評価スクリプトで比較検証)
 - プロンプト運用 (バージョン管理・本番データでの継続改善)
- **生成 AI 活用**: Claude Code (Skills / MCP / エージェント運用 / GitHub Actions 自動レビュー), Codex CLI, Claude Code Managed Agents
- **言語**: TypeScript, Swift, Dart, Python, PHP, Java
- **クラウド**
 - **Cloudflare**
 - コンピュート / 配信: Workers, Pages, Browser Rendering
 - AI: Workers AI, AI Gateway, Vectorize
 - データ / 非同期処理: D1, R2, KV, Queues, Workflows
 - ネットワーク / セキュリティ: DNS, Access, Turnstile, Email Routing, Email Sending
 - **Google Cloud**
 - Cloud Run, GCS, IAM, Cloud Identity, Workload Identity Federation
 - **AWS**
 - Organizations, IAM Identity Center, Amplify, AppSync, Cognito, S3, Route 53
 - **Firebase**
 - Authentication, Firestore, Storage, Analytics, Crashlytics, App Distribution, Remote Config
- **インフラ / IaC**: Terraform, tfint, dotenvx, OIDC
- **サーバーサイド**: REST API 設計, GraphQL 設計, Node.js, Hono, oRPC, PHP, Java
- **データベース**: RDB のテーブル設計, Prisma ORM, Drizzle ORM, PostgreSQL, MySQL, SQLite (Cloudflare D1), Firestore, ベクトル DB (Cloudflare Vectorize)
- **認証 / 認可**: OAuth 2.0, JWT, Better Auth, LINE Login, Cognito, Firebase Authentication, Cloudflare Access, Workload Identity Federation (OIDC)
- **Web アプリ開発**: React, Next.js, Astro, TanStack Start, TanStack Router, TanStack Query, shadcn/ui, Tailwind CSS
- **モバイルアプリ開発**: React Native (Expo), Swift (SwiftUI, UIKit), Flutter
- **アーキテクチャ**
 - モノレポ設計 (Bun workspaces), Clean Architecture, レイヤーアーキテクチャ, MVVM, Redux, Riverpod
 - 独自アーキテクチャ「[Framework-Independent Architecture \(FIA\)](#)」の考案・公開 ([スライド](#) / [YouTube](#))
- **テスト**: Vitest, Playwright, Swift Testing, XCTest, Quick/Nimble, Maestro, Storybook, MSW, @axe-core/playwright
- **コード品質**: Biome, oxlint, oxfmt, ESLint, Prettier, knip, dependency-cruiser, commitlint, lefthook, husky, Renovate
- **CI/CD**: Xcode Cloud, GitHub Actions, Bitrise, release-please, Wrangler
- **SEO / パフォーマンス**
 - Core Web Vitals, Lighthouse, PageSpeed Insights, Google Search Console

- 構造化データ (JSON-LD / Schema.org), OGP 自動生成 (satori / sharp), sitemap / RSS / canonical
- AI クローラー対応 (llms.txt, Content-Signal 対応 robots.txt), Pagefind (静的サイト内全文検索)
- プロジェクト管理: Scrum Master 経験, アジャイル開発 (Jira, Confluence, GitHub Projects, Trello, Zenhub, Notion, Backlog, Linear)
- OSS 開発 (自作 CLI ツール)
 - [ai-chat-md-export](#) (AI チャット履歴の Markdown 変換)
 - [mermaid-markdown-wrap](#) (Mermaid の Markdown ラップ)
 - [issue-linker](#) (GitHub Issue 参照の検証)
 - [readme-i18n-sentinel](#) (README 翻訳の構造検証)
 - 配布: npm / Homebrew / GitHub Actions Marketplace / 実行バイナリ。release-please・Codecov などの品質管理を全ツールで統一

職務経歴

株式会社ZENSHIN (2026年04月 - 現在)

- 2026年
 - [No.11] [株式会社ZENSHIN](#) - CTO / AI コンサルタント / FDE (AI コンサルティング会社)
 - AI コンサルティング / 顧客向け AI ツール開発 (FDE) / 社内 AI システム開発 / コーポレートサイト・社内インフラ

フリーランス (2021年07月 - 2026年03月)

- 2025年
 - [No.10] ショートドラマアプリ開発 - フロントエンドエンジニア (React Native / Next.js) (エンタメ業界 G社)
 - [No.9] NFT ゲームアプリ開発 - Flutter リードエンジニア (WEB3 特化 Sler F社)
- 2024年
 - [No.8] SNS アプリ開発 - iOS リードエンジニア (Sler E社)
- 2023年
 - [No.7] マーケティングリサーチアプリ開発 - iOS リードエンジニア (スタートアップ D社)
- 2022年
 - [No.6] ファンクラブアプリ開発 - iOS エンジニア (メガベンチャー C社)
- 2021年
 - [No.5] ドローン制御アプリ開発 - iOS エンジニア (ドローンベンチャー B社)

外資系ITコンサル A社 (2017年04月 - 2021年06月)

- 2021年
 - [No.4] ショッピングアプリ開発 - Flutter エンジニア
 - [No.3] 飲食店管理アプリ開発 - iOS リードエンジニア
- 2018年 - 2020年
 - [No.2] クレジットカードアプリ iOS アプリ開発 - iOS エンジニア
 - [No.1] クレジットカードアプリ API 開発 - サーバーサイドエンジニア

※ 各案件の詳細は以下のプルダウンから確認可能。

[No.11] 株式会社ZENSHIN - CTO / AI コンサルタント / FDE (AI コンサルティング会社)

チーム体制

- CTO として、AI コンサルティングから社内プロダクト・インフラの開発・運用までを一人で担当

案件概要・担当業務

- 創業フェーズの株式会社ZENSHIN の CTO として、以下 6 つの業務を並行して推進
 - [業務1] AI コンサルティング / 提案活動 (FDE) — AI 活用相談・提案から実装・導入まで支援
 - [業務2] AI マッチングシステム開発 — Cloudflare フルスタックの RAG / MCP / AI エージェント基盤
 - [業務3] コーポレートサイト開発 — <https://www.zenshin-inc.co.jp/> の設計・構築・運用
 - [業務4] 社内インフラ管理 — Terraform によるマルチクラウド IaC
 - [業務5] AI 案件選別システム開発 — 外部案件を LLM で自動評価する社内システム
 - [業務6] 社外向け資料共有基盤の開発 — Cloudflare Access 認証付きの資料配信基盤

[業務1] AI コンサルティング / 提案活動 (FDE)

概要

- システム構築案件の提案活動と、顧問型の「[AI 活用伴走サービス](#)」を一体で推進
- 外部の受発注マッチングプラットフォーム経由の案件獲得も自ら担当し、アポ獲得から商談・提案・見積もりまでを実施
- 提案では AI ツールを活用したデモ・プロトタイプと提案書の作成を主導
- CEO や営業と同席し、要件ヒアリングから提案までを技術面でリード
- 伴走サービスでは、経営者や DX 推進担当者を対象に月 2~4 回のオンライン相談で支援
- 相談・提案で終わらず、FDE として実装・導入まで自ら支援
- 建設業の実案件では、顧客業務に入り込んだ AI ツールを開発し定着まで伴走

経験した技術

- AI ツールを活用したデモ / プロトタイプ / 提案書・見積書の作成
 - v0 (Vercel) / Replit / Lovable / Google AI Studio による UI・アプリの高速プロトタイピング
 - Claude Code / Codex による実装レベルのデモ作成
 - Google Stitch / Figma によるデザインモックアップ作成
 - プロトタイプを組み込んだ提案書・見積書の作成
- AI コンサルティング (AI 活用伴走サービス)
 - 現状ヒアリングと課題整理、業務に適した AI 活用方法の提案
 - ChatGPT / Claude / Gemini などの LLM ツールの使い分け指導、プロンプト・ツール設定の改善支援
 - MCP 化や Claude Code Skills 設計の方針策定支援
- FDE 実案件でのツール開発・導入
 - 建設業向けに、詳細施工計画書 (全 12 章) を Claude Code と共同作成する AI 支援システムを開発し、実案件データで運用
 - Markdown を正本に、pandoc + LibreOffice で Word / PDF を自動生成
 - CAD 図面・Excel などの資料読み取りツール群も整備

取り組み・貢献

- 課題起点の提案
 - お客様の提示するシステム要件をそのまま受けず、「何に困っているか」「理想の姿」を深掘りし、課題定義から提案し直す

- 提案書・見積書は、課題がどう解決されるかと費用対効果（投資回収の見込み）まで含めた意思決定材料として作成
- 実装先行の提案スタイル
 - 商談前に 1〜2 日でプロトタイプを作成し、実現可能性・工数まで詰めた具体性の高い提案を実現
 - 商談中の不明点は、その場で AI を使って調査・回答
 - 提案方針・見積もりは CEO・営業と事前に合意し、複数案（A / B / C）を準備してお客様の反応に応じて出し分け
- 実践知に基づく一貫通貫の FDE スタイル
 - 自社の AI システム開発・エージェント運用で日々実践した知見をコンサルティングに還元
 - 経営層には投資対効果、実装担当者にはプロンプト・ツール設定と、相手のレイヤーで出し分け
 - 助言・提案で終わらず、MCP サーバー構築・Claude Code Skills 整備・AI ワークフロー構築まで自ら実装して顧客に届ける

[業務2] AI マッチングシステム開発（Cloudflare フルスタック AI 基盤）

概要

- Excel 手作業とキーワード検索に依存していた SES のマッチング業務（要員 × 案件）を AI システム化
- Cloudflare Workers 上のフルスタック構成で単独開発・運用
- 案件 1,000 件 × エンジニア 1,000 名の規模から、ベクトル検索と AI 採点により適切なマッチング候補を 5 分以内に提示
- LINE で届く非構造的要員・案件情報を LLM で構造化
- ベクトル検索（RAG）で候補を抽出し、AI エージェントが採点する多層パイプラインを構築
- IT 知識のない営業メンバーが、MCP 経由で Claude Code / Codex からシステムを直接操作
- マッチングした人材の紹介までを、営業の実業務として運用
- 「AI にチャットで聞く」で終わらず、業務プロセスそのものに AI を組み込んだ業務システムとして設計
- 社内スタッフ向け管理画面に加え、toC 向け公開サービスと toB 向けマルチテナント SaaS を同一モノレポで立ち上げ、1 人でマルチプロダクトを開発・運用
 - toC: <https://zenshin-freelance.com/>（LINE 公式アカウントを顧客接点に、登録〜案件提案〜状況確認までを自動化）
 - toB: <https://zenshin-engineer-match.com/>

経験した技術

- RAG / Embedding / ベクトル検索
 - Workers AI (bge-m3, 1024 次元・多言語対応) による Embedding 生成と Cloudflare Vectorize によるコサイン類似度検索
 - メタデータフィルタによる検索母集団の絞り込みと、検索結果の D1 保存によるマッチング候補管理
 - ベクトル検索（一次絞り込み）→ AI エージェント採点（二次精査）の多層マッチングパイプライン設計
- MCP サーバー開発
 - MCP TypeScript SDK (v2) による 70+ ツールを持つ MCP サーバーの実装（Cloudflare Workers 上）
 - Better Auth (Google OAuth) による MCP OAuth 認証・セッション管理で、業務データへのアクセスをユーザー単位でセキュアに制御
 - organization / admin plugin によるマルチテナント認可（ロール階層・招待の許可ドメイン制限）
 - 無人自動化用サービストークンの設計
 - WebMCP（ブラウザ内 MCP）の Origin Trial 先行導入
 - SPA 自身が API 呼び出しを MCP ツールとしてブラウザセッションに公開する構成
 - Claude Code / Codex の両エージェントから同一の MCP サーバーを利用できるマルチエージェント対応
- Cloudflare AI / サーバーレス基盤

- AI Gateway 経由の LLM 統一呼び出し（構造化抽出・Vision 解析・Embedding を同一バインディングで管理）
- Cloudflare Workflows による durable execution（LLM 抽出 → 構造化 → Embedding → マッチング検索の step 単位リトライ）
- Cloudflare Queues + DLQ による非同期マッチング検索
- Browser Rendering + Puppeteer によるスキルシート PDF の画像化 → Vision LLM でのスキルシート解析
- D1 (SQLite) + Drizzle ORM による 160+ マイグレーション管理、KV によるアクセストークンキャッシュ
- ユースケース別の LLM モデル選定
 - モデル評価用のスクリプト群を自作し、Kimi K2 / gpt-oss / GLM / Llama 系を実タスクの精度・コストで比較検証
 - 採否の判断根拠をコードに明文化
 - 構造化抽出・Vision 解析・Embedding などのシーンごとに、精度・レイテンシ・コストのバランスでモデルを使い分け
- AI エージェント運用基盤 (Claude Code Skills)
 - 50+ の Claude Code スキル + 定期ルーチンによる業務自動化（要員・案件の取り込み、マッチング採点、Slack への日次レポート配信）
 - CLAUDE.md / SKILL.md による判断ルール・業務フローの SSoT 化
 - テンプレートを正本に両エージェント向けの指示書を自動生成し、drift を検知
 - 定期実行ジョブを Git 管理の台帳で IaC 的に運用し、実行監査などで自動化を安全に運用
- LINE 公式アカウントによる候補者コミュニケーション基盤
 - LINE Login と Messaging API の userId 同一性を活かした、友だち紐付け不要の本人アカウント連携
 - スタッフ向けの双方向チャット（インボックス・返信待ちキュー・Slack ミラー）
 - AI 採点で高スコアの案件提案を本人の LINE へ自動配信し、マイページでの本人回答までつなぐ
 - 採点の静穏待ち・送信時間帯・連投抑制などの配慮設計
 - Quick Reply による稼働状況の定期ヒアリング自動化
 - 自由文を LLM に解釈させず、ボタン回答からステージ遷移・営業メモ追記まで決定論的に処理
 - MCP 経由で AI エージェントが候補者への返信・案件提案の配信まで実行できる設計
 - 自由文の返信は人間の承認を挟み、送信の監査記録を残す運用
 - LINE の配信障害を検知する不在 watchdog（営業時間帯の受信ゼロ監視）
 - webhook の幂等化・多重リトライによるメッセージの取りこぼし防止
- フルスタック開発
 - Hono + oRPC による型安全な API、React 19 + TanStack Router / Query による SPA
 - TanStack Start (SSR) による公開サイト。API 用 Worker へは Service Binding (Workers 間の内部直接通信) で接続
 - Slack API、Google Drive API との連携
 - Playwright E2E (隔離 D1 + 実 API モード)、Storybook + MSW、PostHog によるイベント計測と LP の A/B テスト基盤
 - Bun workspace によるモノレポ管理 (dependency-cruiser によるレイヤー依存の機械強制)

取り組み・貢献

- Excel 手作業のシステム化と「キーワード検索を超える」マッチングの実現
 - Excel 台帳の手作業とキーワード一致検索に頼っていた業務を、構造化データとベクトル検索のシステムへ置き換え
 - スキルシートの内容まで LLM で構造化し、意味ベースの類似検索を導入
 - キーワード一致ではヒットしない要員 × 案件のマッチングを発見できるように
- 非エンジニアの営業が AI エージェントで実業務を回す仕組みの実現
 - Better Auth の OAuth 認証によるセキュアな MCP 接続を整備

- IT 知識のない営業が、自然言語で要員検索～人材紹介までの実業務を完結できる体制を構築
- MCP ツール設計・スキル整備・権限制御により、エンジニアを介さず営業自身が AI エージェントを日常業務で使いこなす状態を実現
- **AI 採点の決定論的ガードとポリシー運用**
 - 採点ポリシーをバージョン管理し、条件ミスマッチはサーバー側で決定論的にスコア上限を強制（AI の過大評価を防止）
 - 営業の判断・メモを採点ガイドに還元するフィードバックループを運用し、AI 判定と人間判断の乖離を継続的に縮小
- **業務プロセス全体への AI 組み込み・自動化**
 - LINE で届く営業メールの転送テキスト・添付スキルシートを、LLM 構造化 → ベクトル検索 → AI 採点まで自動処理する仕組みを構築
 - スキル + MCP + 定期ルーチンの組み合わせで、レポート配信や AI 自動採点など人手を介さない無人運用を実現
 - 案件収集から AI 採点、本人 LINE への自動提案、本人回答まで、AI の判断が顧客接点に人手を介さず届く自動ループを構築
- **低コスト・高信頼な運用設計**
 - インフラを Cloudflare のみで完結させ、小規模チームでも維持できる運用コストを実現
 - サイレント障害が起きていた非同期処理を Workflows の durable execution へ移行して根治するなど、信頼性を継続的に改善

[業務3] コーポレートサイト開発（Astro + Cloudflare Pages）

概要

- 株式会社ZENSHIN のコーポレートサイト（<https://www.zenshin-inc.co.jp/>）を Astro + Cloudflare Pages 構成で設計・構築・運用
- 技術ブログ（<https://tech.zenshin-inc.co.jp/>）を別アプリとして分離した後、Bun workspaces モノレポへ統合
- 技術ブログのコードは MIT ライセンスで一般公開

経験した技術

- **Astro / Cloudflare Pages**
 - Astro による SSG + Cloudflare Pages Functions による問い合わせ API の実装
 - Tailwind CSS v4 による UI 実装、WCAG AA 準拠のブランドカラーシステムの設計
 - Astro Content Collections による blog / news / works / services のコンテンツ管理
 - Cloudflare Turnstile + Slack Bot Token による問い合わせフォームのスパム対策と通知連携
 - Cloudflare Email Sending + Slack Interactive による採用応募対応の自動化
 - 応募受付から、Slack のボタン → 確認モーダル → お見送りメール送信までを完結
 - `astro:env` による下書き記事の環境別可視化
- **技術ブログ（一般公開リポジトリ）**
 - Astro 7 + Marp による記事・スライドの静的生成、Pagefind による全文検索
 - Pages Functions + Workers KV による閲覧数カウンター API（IP レートリミット付き）
 - Zenn 互換記法プラグインの自作と Zenn クロスポスト運用
 - `llms.txt` / Content-Signal 対応 `robots.txt` による AI クローラー方針の明示
- **SEO / 構造化データ / Core Web Vitals**
 - Schema.org の JSON-LD 埋め込み、OGP / canonical のページ別制御、satori + sharp による OG 画像の動的生成
 - CSS インライン化・画像最適化による LCP 改善、Google Search Console / PageSpeed Insights による計測と継続的改善

- **CI/CD / テスト / 品質**
 - GitHub Actions による Preview / Staging / Production の多段デプロイと `release-please` による自動リリース
 - Vitest / Playwright / `@axe-core/playwright` による単体・E2E・アクセシビリティ検証
 - oxlint / oxfmt / knip / lefthook による静的解析・フォーマット自動化
- **生成 AI 活用**
 - Claude Code Managed Agents のセットアップ、Chrome DevTools MCP でのブラウザ動作検証
 - GitHub Actions による Claude Code 自動レビュー

取り組み・貢献

- **CEO・経営陣と連携したサイト構築**
 - 掲載内容・デザイン方針を CEO や経営陣と相談しながら決定
 - Staging 環境を整備し、段階的に経営陣の意見を取り込む CI/CD サイクルで開発を進めた
- **SEO / Core Web Vitals 改善**
 - CSS インライン化・画像最適化・構造化データ実装により Lighthouse 100/100 を達成・維持
 - Google Search Console と PageSpeed Insights をダッシュボードとして運用し、継続的に改善
- **技術ブログの立ち上げ・運営による採用・ブランディング貢献**
 - タグ / 月別アーカイブ、記事ごとの OGP 自動生成、執筆規約を整備
 - 技術選定・CMS 選定・Astro OGP 生成などの記事を自ら執筆し、採用・ブランディングに活用

[業務4] 社内インフラ管理（Terraform マルチクラウド）

概要

- GCP / Cloudflare / AWS を横断管理する Terraform モノレポを構築・運用
- コーポレートサイトのホスティング基盤、DNS、Zero Trust アクセス制限、アカウント・通知設定などを IaC 化
- クラウドにとどまらず、Google Workspace の組織・グループ権限やブランドアセットまでコード管理の対象を拡大

経験した技術

- **Terraform / マルチクラウド**
 - GCP / Cloudflare / AWS を対象としたマルチモジュール構成の IaC 管理
 - GCS Remote Backend による Terraform state の一元管理
 - Workload Identity Federation (OIDC) による GitHub Actions → GCP の鍵レス認証
 - Cloudflare Access (Zero Trust) + Google OAuth による Staging 環境へのアクセス制限
 - dotenvx による各モジュールの `.env` 暗号化コミットと CI 復号
- **Google Workspace / 組織管理の IaC**
 - Cloud Identity API 経由で Google Workspace のグループ・メンバー・ロールを Terraform 管理
 - 身分グループと役割グループの分離、役員機密と管理者権限の分離設計
 - Google Workspace CLI の OAuth scope を個別列挙する最小権限運用
 - Google Forms の設問定義ファイルを SoT とした宣言的管理
- **AWS / GitHub 連携**
 - AWS Organizations の最小基盤構築（IAM Identity Center の短期認証のみ・アクセスキー不使用）
 - GitHub Actions → AWS の OIDC 連携（必要な read アクションのみの最小権限 role）
 - GitHub App (JWT → installation token) による Organization 内リポジトリ権限の日次自動同期
- **共通基盤 Worker / ブランドアセット管理**
 - 全社共通 Slack App のイベントを Service Binding で各プロダクト Worker へ振り分ける Slack Router Worker の開発

- 設定ファイルを正本に、全サービスのロゴ・favicon・OG 画像を自動生成するブランドアセット基盤の構築
- lock ファイル（コミット SHA + sha256）で各リポジトリへ配布し、byte 単位で検証
- **Cloudflare のセキュリティ / パフォーマンス機能**
 - Smart Tiered Cache / 0-RTT 接続再開 / Speed Brain / Page Shield の Terraform 管理化
 - apex→www 301 Redirect Rules による URL 正規化
 - WAF Custom Rule の導入検討と「現時点では設定しない」意思決定の文書化
- **CI/CD (Terraform) / ガバナンス**
 - GitHub Actions による PR 時の terraform plan 自動実行（変更モジュールのみの差分ベース実行）と tfcmt による PR コメント
 - tflint + lefthook による pre-commit / pre-push の自動検査
 - Renovate によるプロバイダ・依存の自動更新
 - Gmail フィルタ + Slack Email App による通知の Slack 集約
 - GitHub Organization のチーム構成・権限設計

取り組み・貢献

- **属人化の排除と継続運用の仕組み化**
 - 属人的になりがちな社内インフラ構成を徹底的に Terraform で IaC 化
 - Terraform で表現できない設定は README に明示し、CTO が不在でも運用が回る仕組みを整備
- **インフラ 100% IaC 化による安全なワークフロー**
 - コード変更 → PR → terraform plan CI → マージ → ローカル apply の一連のフローを確立
- **DNS 移行プロジェクトの遂行**
 - AWS Route 53 → Cloudflare DNS への段階的移行を Phase 管理で実施し、無停止で本番切替
- **技術的意思決定の文書化**
 - 採用しなかった選択肢（WAF、Cache Rules など）とその理由もリポジトリに残す文化を形成

[業務5] AI 案件選別システム開発 (Cloudflare + LLM)

概要

- 外部の受発注マッチングプラットフォームから届く新着案件を、LLM で自動評価する社内システムを単独で設計・開発・運用
- 応募可否の判断材料（スコア・判定理由・応募文ドラフト）を Slack / Web に配信
- 営業・経営層の「どの案件に応募すべきか」の判断を AI で支援し、最終判断のみ人間が行う設計

経験した技術

- **LLM パイプライン**
 - Slack Events → Cloudflare Queues (DLQ 付き) → Workflows (durable execution) の 3 段非同期パイプラインによる案件処理
 - 案件のライフサイクル通知（商談成立・メッセージ受信・日程調整）を解析する Workflow を追加
 - Slack スレッド上で案件ごとの商談進捗を自動追跡（手動ステータスは上書きしない設計）
 - 案件ページのスクレイピングと決定論的な DOM パース（構造化に AI を使わず、コストと不確実性を排除）
 - Workers AI (Kimi K2.6) + AI Gateway で、スコアリング・応募文・自己推薦文を 1 コールで同時生成
 - PDF / Office ファイルの Markdown 変換 (Workers AI toMarkdown)、Browser Rendering による SPA 参考サイトの取得
- **プロンプト運用**
 - プロンプトのバージョン管理 (25 回超の改修サイクル)

- 本番データでの「AI 判定 vs 人間判断」の乖離分析による継続改善
- Claude Code / Codex 向けの自作スキルによる、プロンプトチューニング・未対応案件の一括選別の半自動化
- AI による実績の捏造を防ぐネガティブリストなどのガードレール設計
- フルスタック / 運用
 - Hono + oRPC による型安全な API、React 19 + TanStack Router / Query による SPA、Drizzle ORM + D1
 - Slack のボタン・モーダルによる判断記録 (HMAC 署名検証)、Cron Triggers による日次リマインド・ステータス監視
 - GitHub Actions による CI/CD (D1 マイグレーション自動適用・Worker Secrets 同期)

取り組み・貢献

- 案件選別業務の自動化
 - 受信から評価・応募文ドラフト生成までを全自動化
 - 営業の案件選別を「全件読む」から「判断するだけ」に省力化
 - エントリー送信はあえて自動化せず、人間の最終判断を挟む安全設計とした
- データ駆動のプロンプト改善運用
 - プロンプト改修の履歴・観点をリポジトリに記録し、判定精度を継続的に改善する運用を確立

【業務6】社外向け資料共有基盤の開発 (Cloudflare Workers)

概要

- 提案書などの社外向け資料を、Cloudflare Access 認証付きで顧客に配信する資料共有基盤
- Cloudflare Workers + Static Assets で単独開発・運用
- 案件担当者はディレクトリに資料を置くだけで顧客への URL 共有が完結する仕組み

経験した技術

- Worker の HTMLRewriter による共通フレーム挿入と Markdown の実行時 HTML 化 (ビルドレス設計)
- GitHub App による別リポジトリ資料の allowlist 型ライブ proxy、Cache API の stale-while-revalidate
- access 設定ファイルを SSoT とした Cloudflare Access ポリシーの GitHub Actions 自動同期
- 誤公開を防ぐ二重の機械的チェック (fail-safe 設計)、機密設定ファイルの配信禁止・ダウンロード機能の非搭載など認証境界の多重防御
- git 履歴からの目次メタデータ自動生成
- Cloudflare / Chrome DevTools の MCP による本番ログ調査・Access 越しの UI 検証

取り組み・貢献

- 顧客資料の共有を「URL を渡すだけ」に統一
- 認証・公開範囲の管理を機械強制し、非エンジニアの案件担当者でも安全に使える基盤を実現

開発環境

フロントエンド

- Astro, React 19, TanStack Start, TanStack Router, TanStack Query, shadcn/ui, TypeScript, Tailwind CSS v4, Bun, Astro Content Collections, Marp, Pagefind, satori, sharp

バックエンド / AI システム

- Hono, oRPC, Drizzle ORM, Zod, Better Auth (organization / admin plugin), @modelcontextprotocol/sdk v2 (MCP サーバー開発), WebMCP
- Cloudflare Workers AI (bge-m3, Kimi K2.6, toMarkdown), AI Gateway, Vectorize, Workflows, Queues, Browser Rendering (Puppeteer), Email Routing, Email Sending, HTMLRewriter

- LINE Messaging API, Slack API (Events / ボタン・モーダル), Google Drive API, GitHub App 連携

FDE 案件ツール開発

- Python (uv), pandoc + LibreOffice (Word / PDF 自動生成)

クラウド / インフラ

- Cloudflare (Workers, Pages, Pages Functions, D1, R2, KV, DNS, Turnstile, Access), GCP (Workload Identity Federation, GCS, IAM, Cloud Identity), AWS (Organizations, IAM Identity Center), Terraform, tflint, dotenvx, mise

CI/CD

- GitHub Actions, release-please, Wrangler, tfcmt, Renovate

テスト / 品質

- Vitest, Playwright, Storybook, MSW, @axe-core/playwright, oxlint, oxfmt, knip, syncpack, dependency-cruiser, lefthook, commitlint

AI ツール (提案活動・開発)

- v0 (Vercel), Replit, Lovable, Google AI Studio, Google Stitch, Claude Code (Skills / MCP / 定期ルーチン), Codex, Claude Code Managed Agents, Chrome DevTools MCP

分析 / モニタリング

- Google Search Console, PageSpeed Insights, Cloudflare Analytics, PostHog

開発ツール

- VSCode, Figma

[No.10] ショートドラマアプリ開発 - フロントエンドエンジニア (React Native / Next.js)

チーム体制

- 案件全体人数 : 6名
 - フロントエンド (iOS + Android + Web) : 2名 (担当)

案件概要・担当業務

- 0 → 1 直後のグロースフェーズにおけるショートドラマアプリの開発
- React Native (Expo) による iOS / Android 同時開発を担当し、並行して Next.js による Web 版の開発も実施

経験した技術

- **React Native (Expo) / Next.js**
 - Monorepo 構成による iOS / Android / Web のクロスプラットフォーム開発
 - Expo SDK のメジャーバージョンアップ対応 (v52→v53、v53→v54、v54→v55)
 - Solito によるモバイル (Expo) / Web (Next.js) 間のコード共有
 - Tamagui を用いたプラットフォーム間の UI 統一
 - expo-iap によるアプリ内課金の実装
 - Rive を用いたアニメーションの組み込み
 - Next.js による Web アプリケーション開発
 - TanStack Query によるデータフェッチング・キャッシング
 - Crisp SDK を用いたカスタマーサポート用チャット機能の実装
- **開発プロセス改善**
 - Storybook による UI コンポーネントカタログの作成

- Maestro によるモバイルアプリでの E2E テストの導入
- Mise の導入による開発環境のセットアップの簡略化とバージョンの統一
- **AI・MCP活用**
 - Figma MCP を活用した Figma デザインからの実装
 - Storybook でコンポーネントを作成し、Google DevTools MCP / Playwright MCP で Claude Code による検証
 - GitHub Actions による Claude Code 自動レビューの仕組みの構築

取り組み・貢献

- **ビジネス指標に基づくグロース施策の企画・実装**
 - ARPU・CTR・LTV などの KPI を CEO と連携しながら追い、ユーザー獲得コスト vs LTV を意識した施策設計を行った
 - ファネル分析で離脱ポイントを特定し、課金導線の改善によりサブスクの契約者数や課金アイテムの購入数を増加させた
 - ガチャ・ミッション機能・無料開放施策などの設計・実装を通じて、主要 KPI の改善に貢献した
 - オファークロール連携を導入し、収益チャネルを拡大した
- **高速な PDCA サイクルの実践**
 - 施策立案から1週間でリリースする高速な PDCA サイクルをスプリントごとに繰り返し回した
 - A/B テストによる効果検証を行い、季節要因や曜日パターンなど複数の変数がある中で施策単体の効果を正しく測定した
 - 効果が見込めない施策は即座に機能削除し、効果がないという事実も意思決定に活かした
- **デザイナーとの協業**
 - 表示内容や画面遷移の導線、コンポーネントの色や配置といった UI/UX の仕様にまで踏み込んで改善案を提案した
 - 仮実装を素早く共有し、デザイナーと議論しながらリリース前に何度もブラッシュアップを重ねた
 - リリースごとの KPI の結果をデザイナーと共有し、データに基づいた改善提案を繰り返すことで提案の精度を高めていった

開発環境

React Native (Expo) / Next.js

- **アーキテクチャ:**
 - Monorepo 構成 (Expo + Next.js)
- **主要ライブラリ:**
 - TanStack Query, ts-proto, Tamagui, Solito, Expo Modules API (Swift / Kotlin)
- **コード品質・テスト:**
 - Storybook, ESLint, Prettier, Vitest, Maestro

Firebase / Google Cloud

- Firebase (Auth, Distribution, Remote Config, Analytics)
- Google Cloud (Cloud Run)

分析ツール

- BytePlus DI, Looker Studio, Adjust

CI/CD

- GitHub Actions

プロジェクト管理

- GitHub Projects, Notion, Linear

開発ツール

- VSCode, Android Studio, Xcode, GitHub Copilot, Claude Code, Codex, Gemini

デザインツール

- Figma

[No.9] NFT ゲームアプリ開発 - Flutter リードエンジニア (Flutter)

チーム体制

- 体制
 - PdM : 1名
 - PM : 1名
 - デザイナー : 1名
 - サーバーサイドエンジニア : 2名
 - Flutter エンジニア : 1名 (担当)

案件概要・担当業務

- 0 -> 1 フェーズでの NFT ゲームアプリの開発における Flutter エンジニアを担当
- 唯一の Flutter エンジニアとして、アーキテクチャ考案・ライブラリ選定からすべての実装までを担当
- PM・デザイナー・サーバーサイドチームとの仕様調整も担当

経験した技術

- **Flutter**
 - Riverpod と Hooks を用いた状態管理
 - Google Maps API の活用と google_maps_flutter , geolocator を用いた地図表示と位置情報の取得
 - go_router を用いた画面遷移の実装
 - openapi_generator を用いた API クライアントコードの自動生成
 - dio を用いた HTTP 通信およびインターセプターによる JWT 認証の実装
 - flutter_secure_storage を用いたセキュアなデータ保存
 - permission_handler を用いた位置情報取得、写真撮影、写真フォルダへのアクセスの実装
 - slang による多言語対応
 - pedantic_mono によるコード品質の向上
 - ThemeData よりデザインシステムの実装
 - fvm による Flutter バージョンの管理
- **開発プロセス改善**
 - Prism を活用した API モックサーバーの構築
 - Lefthook による pre-commit 時の静的解析実行

取り組み・貢献

- **開発体制の改善活動**
 - GitHub Projects を活用したスクラムボードを作成し、タスクの進捗状況を可視化した
 - デイリーを開催し、毎日、メンバー間での情報共有と開発プロセスの改善を行った
 - バグの発見から修正までのプロセスを整備して、それをチーム内で運用した
- **API インターフェース設計と UI 先行開発**
 - サーバーサイドの Pull Request をレビューし、開発段階で API インターフェースの改善点をフィードバックを行った

- OpenAPI 形式の yaml ファイルから Prism でのモックサーバーでの開発環境を整備して、UI の先行開発を実施した
- デザインシステムの導入
 - デザイナーと協力してデザインシステムを設計し、デザインの一貫性を実現した
 - デザインシステムを Flutter の `ThemeData` を通じて定義し、UI の実装コードを削減した

開発環境

Flutter

- アーキテクチャ:
 - Riverpod + Hooks による状態管理
- 主要ライブラリ:
 - go_router, dio, slang, permission_handler, flutter_secure_storage, pedantic_mono, freezed, google_maps_flutter, geolocator, openapi_generator, fvm

CI/CD

- GitHub Actions

プロジェクト管理

- GitHub Projects, Notion, Slack

開発ツール

- VSCode, Android Studio, Xcode, GitHub Copilot, ChatGPT

デザインツール

- Figma

[No.8] SNS アプリ開発 - iOS リードエンジニア (Swift)

チーム体制

- 案件全体人数：約10名
 - iOS エンジニア：1名（担当）

案件概要・担当業務

- 0 → 1 フェーズでの SNS アプリ開発の立ち上げ案件
- 唯一の iOS エンジニアとして、アーキテクチャ考案・ライブラリ選定・CI/CD 構築からすべての実装までを担当
- PM・デザイナー・サーバーサイドチームとの仕様調整も担当

経験した技術

- **Swift**
 - Xcode 16 Beta での Strict Concurrency を含む Swift 6 対応
 - AVFoundation を活用した録音/再生の機能実装
 - [WhisperKit](#), [Speech](#) SDK を活用した音声データの文字起こしの実装
- 開発プロセス改善
 - [Swift OpenAPI Generator](#) による API 通信処理の自動生成の GitHub Actions パイプラインの構築
 - [Swagger UI Action](#) を用いた API 仕様書の自動生成の GitHub Actions パイプラインの構築
 - [tbls](#) を用いた MySQL のテーブル定義書の自動生成の GitHub Actions パイプラインの構築
 - [pixelmatch](#) による View のスナップショットの差分検出の実装

取り組み・貢献

- **アジャイル開発**
 - テスタブルなアーキテクチャの導入:
 - モックで API レスポンスを差し替えられるアーキテクチャを導入し、API 提供前から実装を可能にした
 - デバッグ画面の作成:
 - iOS アプリに検証用のデバッグ画面を作成し、新機能や View の早期検証を可能にした
 - Docs as Code の導入:
 - [Swagger UI Action](#) や [tbls](#) によるドキュメント生成方法を調査して、サーバーサイドチームに展開した
- **CI/CD 環境の構築**
 - Xcode Cloud 導入:
 - Xcode Cloud で PR マージをトリガーに TestFlight 配信を自動化し、新機能の迅速な検証を可能にした
 - API インターフェース変更の自動 Pull Request 作成:
 - OpenAPI の変更をトリガーに、iOS リポジトリへ自動 Pull Request を作成する GitHub Actions を構築した
 - スナップショット差分テスト:
 - View のスナップショット差分テスト環境を構築し、不具合の早期発見を実現した
- **iOS メンバーの増員や引き継ぎを見越した GitHub 管理**
 - ドキュメント整備:
 - 環境構築手順、ライブラリ選定理由、アーキテクチャ、CI/CD 構成図、ブランチ戦略などを README に記載した
 - プロジェクト管理:
 - リリースノート、タグ、マイルストーン、GitHub Projects を整備し、タスクの進捗を時系列で振り返れるように管理した
- **実装・最新技術**
 - 実装:
 - ワイヤフレーム段階でのデザインを基に iOS アプリを実装し、実装の課題や仕様の課題を早期発見し、チームへ共有した
 - コード生成:
 - View 層や UseCase 層のテストコードを含めたボイラープレートコードは [Sourcery](#) や [Mockolo](#) によって自動生成し、開発効率を高めた
 - Swift 5 → Swift 6 への移行:
 - 早い段階から Beta 版 Xcode を用いて Swift 6 への移行を検証し、大きなトラブルなくスムーズに移行を完了した

開発環境

Swift

- **アーキテクチャ:**
 - Clean Architecture x Swift Package Manager でのマルチモジュール構成
- **Swift 標準 SDK & API:**
 - SwiftUI, Swift Package Manager, Swift Concurrency, Combine, AVFoundation, Speech, Swift Testing, String Catalogs, Swift OpenAPI Generator

CI/CD

- Xcode Cloud, GitHub Actions, Renovate

プロジェクト管理

- GitHub Projects, Notion, Backlog

デザインツール

- Figma

[No.7] マーケティングリサーチアプリ開発 - iOS リードエンジニア (Swift)

チーム体制

- 案件全体人数：約15名
 - iOS エンジニア：3名 (iOS リードエンジニア担当)

案件概要・担当業務

- スタートアップ企業の 0 → 1 フェーズでのマーケティングリサーチサービスの立ち上げ案件
- toC 向けコンテンツ配信アプリと toB 向け BI ツールの 2 サービス構成で、iOS チームのリードエンジニアを担当

経験した技術

- **Swift**
 - iOS16 以上を対象 OS とした SwiftUI での画面開発
 - Clean Architecture x Swift Package Manager でのマルチモジュール構成の構築
 - Xcode Cloud での CI/CD 環境の構築
 - Protocol Buffers に対応した [SwiftProtobuf](#) のライブラリを用いたデータ連携
 - async/await, AsyncStream, TaskGroup, Actor などを用いた Swift Concurrency による非同期処理のハンドリング
 - [AWS Amplify SDK](#) を用いた Cognito での SMS での認証・認可、AppSync による GraphQL 疎通、Pinpoint によるログイベント送信、S3 とのデータ連携
 - デザインシステムを活用した画面実装
 - AVFoundation を用いた動画の再生
 - ReplayKit を用いた画面のレコーディング
 - 視線や感情の時系列データの Combine を用いたハンドリング
 - JavaScript を用いたアプリ内 WebView のイベントハンドリング
- **開発プロセス改善**
 - リリース tag・リリースノート・レビュー追加などを自動化する GitHub Actions Workflow の実装
 - [Renovate](#) によるライブラリの自動更新 PR の作成の環境構築
 - [Periphery](#) による Swift コードの不要なコードの静的解析
 - Swift-DocC による iOS アプリのドメイン層のドキュメント化
 - [Mockolo](#) によるテスト用の Mock の自動生成
 - GitHub Copilot, ChatGPT の活用

取り組み・貢献

- 0 → 1 フェーズの iOS リードとして、アーキテクチャ・ライブラリ選定、ブランチ戦略、リリース手順の確立を担当した
- CI/CD 環境の構築や、iOS チームのスクラムボード運用の設計も行った
- チームに経験者のいない AWS Amplify SDK や SwiftProtobuf は、サンプルアプリで先行検証してチームに展開し、採用につなげた
- 視線分析・感情分析の SDK を、入れ替えても影響範囲が最小になるアーキテクチャで組み込んだ
- PdM・デザイナー・サーバーサイド・データ分析チームと、仕様やデータ連携インターフェースを調整した
- iOS チームの issue 運用を担当し、メンバーのタスクが途切れないよう先回りして行動し続けた

開発環境

Swift

- アーキテクチャ:
 - VIPER ベースの Clean Architecture x Swift Package Manager でのマルチモジュール構成
- **Swift 標準 SDK & API:**
 - SwiftUI, Swift Package Manager, Swift Concurrency, Combine, Swift-DocC, AVFoundation, Core ML, WebKit, ReplayKit, Logger
- サードパーティ製 **SDK:**
 - SwiftProtobuf, Firebase, Amplify, Nimble/Quick, LicensesPlugin, PhoneNumberKit, DeviceKit, SwiftFormat, SwiftGen, Lottie, Mockolo, Mint, Periphery

クラウド連携

- **AWS Amplify:**
 - AppSync (GraphQL), Cognito, S3, Pinpoint
- **Firebase:**
 - Crashlytics

CI/CD

- Xcode Cloud, GitHub Actions, Renovate

プロジェクト管理

- GitHub Projects, Notion, Backlog

インターフェース共有

- Protocol Buffers, Swagger

デザインツール

- Figma

[No.6] ファンクラブアプリ開発 - iOS エンジニア (Swift)

チーム体制

- 案件全体人数 : 約30名
 - iOS エンジニア : 5名 (担当)

案件概要・担当業務

- アーティストのファンクラブアプリにおけるスタンプラリー機能および景品交換の機能の開発を行なった
- デザイナーとの仕様の調整、見積もり、実装、レビュー、バグ修正を行なった

経験した技術

- Redux ベースのアーキテクチャ (VueFlux) での状態管理
- デザインシステムに沿った UI の実装

取り組み・貢献

- デザイナー、Android、Web フロントのエンジニアとコミュニケーションを取りながら、プラットフォーム間で仕様に大きな差がでないように開発した

開発環境

Swift

- アーキテクチャ:
 - Redux ベースのアーキテクチャ
- **Swift 標準 SDK & API:**
 - UIKit, AVFoundation
- サードパーティ製 **SDK:**
 - Carbon, VueFlux, ReactiveSwift, XcodeGen, Quick/Nimble, APIKit, CocoaPods, Carthage, Lottie

クラウド連携

- **Firebase:**
 - Crashlytics

CI/CD

- CircleCI, Fastlane

プロジェクト管理

- Wrike, Kibela

インターフェース共有

- Protocol Buffers, Swagger

デザインツール

- Figma
-

[No.5] ドローン制御アプリ開発 - iOS エンジニア (Swift)

チーム体制

- 案件全体人数 : 約15名
 - iOS エンジニア : 6名 (担当)

案件概要・担当業務

- BtoB 向けドローン制御アプリで、飛行の安定性改善・複数社ドローン対応・画面開発などを担当した
- アーキテクチャの検討、見積もり、実装、レビュー、バグ修正を行なった

経験した技術

- **Swift**
 - アーキテクチャの検討
 - Clean Architecture での実装
 - SwiftUI・UIKit x Combine を用いた画面実装
 - Swift Concurrency を用いた非同期処理の実装
 - Firebase Crashlytics、Xcode Organizer を用いたバグの原因調査
 - Logger API を用いたログ出力
 - Quick/Nimble ライブラリを用いた可読性の高いテストコードの記述
 - Mock を活用したテストコードの記述
 - iPad サイズ対応のアプリの実装
- **IoT**
 - 外部ライブラリを用いたドローンの制御の Swift での実装
 - PID 制御などの制御工学の理解と適切な制御モデルの Swift での実装

- RoS(Robot Operating System) 環境の活用

取り組み・貢献

- Clean Architecture の採用により、UI を変えずにライブラリを差し替え、レイヤーごとの独立テストを実現した
- UIKit や Delegate パターンでの既存実装を、SwiftUI、Combine、Swift Concurrency といった新しい技術でリファクタリングした
- 以下のようなチームの運用の改善に積極的に取り組んだ
 - 見積会の実施
 - レトロスペクティブの実施
 - 開発チーム朝ハドル会の実施
 - プロダクトバックログを開発者が着手可能であることを表す「Ready」の概念の導入
 - Pull Request 提出から Merge までの運用ルールの見直し
 - リリースブランチ運用の見直し
 - デイリー前の Slack リマインダーの設定
 - デイリーでの相談事項の事前エントリー制の導入
 - Firebase Crashlytics 運用の見直し

開発環境

Swift

- **アーキテクチャ:**
 - VIPER ベースの Clean Architecture
- **Swift 標準 SDK & API:**
 - SwiftUI, UIKit, Combine, Swift Concurrency, Logger, MetricKit
- **サードパーティ製 SDK:**
 - Realm, Quick/Nimble, APIKit, CocoaPods, Carthage

クラウド連携

- **Firebase:**
 - Crashlytics, Analytics

CI/CD

- Bitrise, Fastlane

プロジェクト管理

- Zenhub

デザインツール

- Figma

[No.4] ショッピングアプリ開発 - Flutter エンジニア (Flutter)

チーム体制

- 案件全体人数 : 2名
 - Flutter エンジニア : 1名 (担当)
 - デザイナー : 1名

案件概要・担当業務

- Flutter での iOS・Android クロスプラットフォーム開発を採用したショッピングアプリのデモアプリの開発を担当
- Flutter でのフロントエンド実装から Firebase を活用したバックエンド実装まで、すべて一人で行った

経験した技術

- **Flutter**
 - Provider による状態管理
- **Firebase**
 - Authentication による認証
 - Firestore によるデータの永続化、NoSQL DB 設計
 - Storage への画像データの永続化
 - Crashlytics によるクラッシュ報告管理
 - App Distribution による iOS・Android のアプリ配布
 - Analytics による KPI 指標の集計
 - Google Maps API での地図活用
- **CI/CD**
 - Codemagic での iOS・Android のアプリ配布の自動化
 - Fastlane から App Distribution への配布

取り組み・貢献

- Firebase を活用したサーバーレス構成でのモバイルバックエンドサービスの設計・実装を行った
- Flutter による Android 対応（マテリアルデザイン、Google Play ストアでの配信）を経験した

開発環境

Flutter

- Provider

クラウド連携

- Firebase:
 - Authentication, Firestore, Storage, Crashlytics, App Distribution, Analytics
- Google Maps API

CI/CD

- Codemagic, Fastlane

デザインツール

- Adobe XD

[No.3] 飲食店管理アプリ開発 - iOS リードエンジニア (Swift)

チーム体制

- 案件全体人数：約10名
 - iOS エンジニア：3名（リードエンジニア担当）

案件概要・担当業務

- BtoB 向け飲食店管理モバイルアプリの MVP アプリの作成
- iOS リードエンジニアとして、要件の調整やサーバーサイドチームとの API インターフェースの検討などを行った
- Scrum Master も兼任した

経験した技術

- **Swift**
 - SwiftUI での画面実装
 - Codable プロトコルを用いた JSON の変換
 - TestFlight によるアプリ配信
 - アーキテクチャ、ディレクトリ構成の検討
- **Scrum Master**
 - スクラムボードの設計
 - 会議のファシリテーション

取り組み・貢献

- 決められた要件をただ実装するだけではなく、お客様やデザイナーによりよい仕様やデザインを提案した
- スクラムボードの扱いやコードレビュー方法などを、レトロスペクティブを待たずチーム内で相談し常に改善した

開発環境

Swift

- **アーキテクチャ:**
 - MVVM
- **Swift 標準 SDK & API:**
 - SwiftUI
- **サードパーティ製 SDK:**
 - SwiftLint

プロジェクト管理

- Zenhub, Trello

デザインツール

- Adobe XD

[No.2] クレジットカードアプリ iOS アプリ開発 - iOS エンジニア (Swift)

チーム体制

- 案件全体人数 : 約20名
 - iOS エンジニア : 4-5名 (担当)

案件概要・担当業務

- BtoC 向けのクレジットカードアプリの iOS アプリの開発における見積もり、実装、テスト、レビュー、バグ修正を担当した
- メインはサーバーサイドチームの担当であったが、作業の手が空いたり、iOS チームの負荷が上がったときに iOS チームを担当した

経験した技術

- UIKit での画面実装
- API 疎通
- Realm でのデータ永続化
- XCTest でのテストコード実装
- MVVM での実装
- Delegate パターンの実装

- Human Interface Guidelines に基づいた UI 実装
- Moneytree LINK SDK といったサードパーティー製のライブラリの組み込み

開発環境

Swift

- アーキテクチャ:
 - MVVM
- Swift 標準 SDK & API:
 - UIKit
- サードパーティー製 SDK:
 - CocoaPods, Carthage, Realm, Moneytree LINK SDK

通信キャプチャ

- mitmproxy

プロジェクト管理

- Jira, Confluence, Trello

デザインツール

- Sketch, InVision

[No.1] クレジットカードアプリ API 開発 - サーバーサイドエンジニア (PHP)

チーム体制

- 案件全体人数 : 約20名
 - サーバーサイドエンジニア : 4名 (担当)

案件概要・担当業務

- BtoC 向けクレジットカード明細管理アプリのリニューアルに伴い、API やバッチの開発
- サプリードディベロッパーとして、お客様向け説明資料の作成、設計、見積もり、実装、テスト、レビューを担当
- チーム結成から約 2 年半、リリースしたシステムを継続的にアップデートした

経験した技術

- **API 設計/開発**
 - PHP での API 設計・開発
 - MySQL での DB 設計・開発
 - OAuth2.0 での認証・認可の実装
- **バッチ設計/開発**
 - Java でのバッチの設計・開発
- **テスト**
 - API の単体・結合テストの設計と実装
 - Postman での API テストの自動化
 - JMeter での負荷テスト
- **ドキュメンテーション**
 - OpenAPI (Swagger) でのインターフェース設計・共有
 - PlantUML での設計

取り組み・貢献

- モバイルアプリケーションのバックエンド開発における設計、実装、テスト、リリース、運用までのフルライフサイクルを経験できた
- iOS チームと兼任していたため、モバイルアプリからの視点を API のインターフェースの設計に取り込むことができた
- API の結合テストを Postman によって自動化することで、少ない工数で網羅的に繰り返しテストを実施し、品質を担保することができた

開発環境

使用言語

- PHP (CodeIgniter)
- Java

プロジェクト管理

- Jira, Confluence, Trello

テストツール

- Postman, JMeter

ドキュメンテーション

- OpenAPI, PlantUML, draw.io
-